Andrea Colombo - Algorithms for Massive Datasets course project - UniMi A.A 2025/2026

Andrea Colombo

Abstract — This report contains the documentation related to the project submission for the *Algorithms for Massive Datasets* course. Taught by *Dario Malchiodi* at *Università degli studi di Milano*.
We present the implementation of two stream analysis algorithms: Flajolet-Martin and Alon-Matias-Szegedy, starting from the theory behind them and going all the way through the implementation choices and experimental results. Additionally, we also present a bloom filter implementation and evaluate the false positive rate at different configurations of bits per element and number of hash functions used.

All the code referenced in this report can be found in the jupyter notebook submitted alongside this file.

Table of Contents

1 Introduction

The analysis of massive datasets has become an increasingly important problem in the later years. These scenarios pose significant challenges due to the sheer amount of data to be processed; in most cases, storing the entire dataset in memory is not feasible.
Stream analysis algorithms address this issue by processing the dataset sequentially, using limited amounts of memory and providing estimations about the stream properties we are interested in. Since these algorithms do not need to store the entirety of the stream in memory, they are used across a multitude of domains and hardware, ranging from huge server rigs to tiny embedded microcontrollers.

The algorithms and tasks we are going to explore are the following:

- Using the *Flajolet-Martin Algorithm* [1] to produce an estimation of the unique userIDs present in the dataset. The full description of the algorithm and the implementation choices can be found in Section 4.
- Using the Alon–Matias–Szegedy [2] algorithm to estimate the second moment of the article sections that have been commented by the users.

Additionally, we present a widely used probabilistic data structure, the bloom filter [3], for the following task:

- Evaluating the performance of a bloom filter implementation Section 6, focussing on how the false positive rate is affected by the configuration parameters m and k (bits per element and number of hash functions used). This implementation was tested by combining a stream of unique *UserIDs* (taken from the original dataset) and an additional stream of procedurally generated IDs.

1.1 Development Environment

This project was developed using *Google Colab* as the main computation environment. The Jupyter Notebook submitted alongside this project may require some modifications before being run in a local environment.
Below is a brief description of the computational resources available on the CoLab runtime and the versions of the main libraries used in this project:

- **CPU**: Intel Xeon CPU with 2 vCPUs (virtual CPUs) and 13GB of RAM.
- **Python Version**: 3.13.15 (built with GCC 11.4.0)
- **PySpark Version**: 4.0.4
- **xxHash Version**: 4.0.1

Please note that these versions are correct at the time of writing: 2026-09-10

The notebook will take care of installing all the required libraries and the dataset automatically.

2 Dataset Description

The dataset used for this project is the **New York Times Articles & Comments (2020)** [4] dataset, freely available and licensed under the **CC-BY-NC-SA-4.0** license.

The dataset, once downloaded, has a size of approximately **6.15 GBs** and presents itself in the form of multiple files:

- **nyt-articles-2020.csv**: Contains all the articles published in 2020 by the NYT.
- **nyt-comments-2020.csv**: this file contains all the comments relative to the articles found in *nyt-articles-2020.csv*.
- **nyt-comments-part0.csv .. nyt-comments-part9.csv**: these files simply contain the data found in *nyt-comments-2020.csv* split in 10 different partitions.
- **test.csv**: Not relevant for our use case
- **train.csv**: Not relevant for our use case

The version of the dataset used for the project is the following:

- Version 6 on Kaggle.

The dataset was last accessed on 2026-09-10

2.1 Preprocessing Techniques

During the preprocessing phase of the project we discarded all the columns that are not relevant for our use case, in particular:

- For the Flajolet-Martin portion of the project (Section 4), we discarded all the columns except the *UserID* and, since all the fields inside the schema are tagged as **nullable**, we excluded all the null entries to avoid using garbage data.
- For the AMS portion of the project, we used the already computed dataframe from the FM algorithm and combined that with the dataset containing all the articles. This was done to extract a stream of articles sections that will be used to estimate the second moment. Since the users were already (optionally) subsampling in the first point, no additional subsampling was applied.
- For the Bloom Filter implementation, the process was a little different, we first gathered the article section that received the most comments (in our case: *Opinion*) and we extracted the set of all unique userIDs that commented those articles at least once.
  After that, we procedurally generated a set of fake *UserIds* that was then used to test the false positive rate of the bloom filter implementation at different values of k and m (see Section 6)

2.2 Subsampling

In order to ensure a reasonable execution time, we introduced a method that allows the user to load only a part of the dataset instead of the whole. This method can be tweaked by modifying the **SAMPLING_PROPORTION** (see: Section 3) variable inside the notebook provided alongside

this document.
By default, the submitted notebook only uses 30% of the original dataset.

3 System Configuration

The python notebook submitted with this project can be configured with the following set of variables:

- **ENABLE_LOGGING**: enables additional prints during the notebook execution
- **ENABLE_ADDITIONAL_EXPERIMENTS**: enables extra portions of the notebook that are mainly used for comparative studies and for generating the plots seen in the report (very time consuming).
- **SAMPLING_PROPORTION**: How much of the whole dataset we want to use for the current run, valid if in range $(0, 1]$.
- **RAND_SEED**: Random number generator seeding for reproducibility.
- **FM_NUM_HASHES**: Number of hash functions to use for the Flajolet-Martin implementation.
- **FM_GROUP_SIZES**: Number of FM registers averaged together (raw values, before exponentiating) to form a single group estimate, as part of the stochastic averaging technique described in Section 4.
- **AMS_STORED_VARS**: Number of variables that the AMS implementation keeps track of inside the reservoir.
- **BLOOM_FILTER_N_HASHES**: Number of hash functions used for each element in the bloom filter implementation (used for the single run only, not for the optional cells enabled with *ENABLE_ADDITIONAL_EXPERIMENTS*).
- **BLOOM_FILTER_N_BITS**: Number of total bits used in the bloom filter (used for the single run only, not for the optional cells enabled with *ENABLE_ADDITIONAL_EXPERIMENTS*).
- **BLOOM_FILTER_FAKE_IDS_PROPORTION**: Used for generating a stream of fake *UserIDs* that is used inside the bloom filter portion of the notebook. The proportion represents how many fake ids we want to generate for each real one

4 Flajolet–Martin Algorithm

First introduced in 1985 [1], the Flajolet-Martin algorithm is a probabilistic streaming algorithm that aims at estimating the number of distinct elements through the use of hash functions.
The core idea is to leverage two very important properties of hash functions:

- **Determinism**: the hash function always maps the same value to the same result
- **Uniform Distribution**: the hashed values are uniformly distributed over the binary space

But why do we focus on the number of trailing zeros (tail length) of the hashed value?
The probability that a single hash value ends with n trailing zeros is $\frac{1}{2^n}$.
From this formula we can derive that:

- The probability of a single hash value not having n trailing zeros is $1 - \frac{1}{2^n} = 1 - 2^{-n}$

- The probability of **none** of k hash values having n trailing zeros is $\left(1 - \frac{1}{2^n}\right)^k$. This can be approximated to $e^{-k2^{-n}}$ for very large numbers.

Therefore, if the maximum number of trailing zeros is $R_{\max}$, it implies that we have seen approximatively $2^{R_{\max}}$ unique elements so far inside the stream.

4.1 The Outlier Problem

Using a single hash function has one big issue, since we are taking the maximum number of trailing zeros from a single source, we are susceptible to outliers.

The solution is simply using multiple hash functions and compute the median of their results. This has been verified to provide more accurate results but it still has one problem, it always results in estimates that are power of 2.
In order to fix this issue, we compute the average of the median estimations we just computed.

4.2 Space/Time Complexity

This algorithm has a time complexity of $O(kn)$ where n is the length of the stream and k is the constant value that refers to the computational cost of the hash functions and update procedure. The space complexity is $O(k)$ instead, we only need to store a constant amount of information needed to update the trailing zeros counts; the precise memory usage depends on how many hash function we decide to use for the algorithm.

4.3 Implementation Details

4.3.1 Hash Function Choice

The hash function used in the submitted implementation is xxhash, an extremely fast non cryptographic hashing algorithm. In particular, this implementation uses the xxh3 64 bits version, xxhash is one of the most used hashing algorithms for non cryptographic uses in real world use cases (PySpark uses it too).

4.3.2 Algorithm implementation

The portion of the algorithm responsible for computing the tail lengths and keeping track of the maximum values can be described with 3 simple functions. Each partition yields a sequence of length *FM_NUM_HASHES* that is then used to compute the final estimation.

```python
def ctz(x):
    if x == 0:
        return 64
    return (x & -x).bit_length() - 1

def hash64bits(value, seed):
    return xxhash.xxh3_64(value.to_bytes(8, "little"), seed).intdigest()
```

```
def fm_partition(it):
    registers = [0] * FM_NUM_HASHES
    for user in it:
        for i in range(FM_NUM_HASHES):
            bits = hash64bits(user, i)
            r = ctz(bits)
            if r > registers[i]:
                registers[i] = r

    yield registers
```

4.4 Experimental Results

We ran the Flajolet-Martin estimator with $FM\_NUM\_HASHES = 16$ (see Section 3) hash functions against the stream of unique commentor *userIDs*, and compared the resulting estimate against the exact distinct count (computed directly via aggregation) and against Spark's built-in HyperLogLog estimator, used here as a reference for how a more refined cardinality estimator performs on the same data.

Method	Estimate	Relative error
Exact count	218,567	Not Applicable
FM (naive, no grouping)	338,902	55.1%
FM (grouped, $FM\_GROUP\_SIZES = 4$)	262,311	20.0%
Spark HyperLogLog	238,510	9.1%

As expected from the stochastic averaging technique described in Section 4, grouping the registers before exponentiating substantially reduces the estimation error compared to taking the median of the raw hash function estimates:

- the grouped estimate's error (20.0%) is roughly a third of the naive estimate's error (55.1%). This confirms that averaging the register values within each group, rather than averaging the resulting 2^R estimates directly, is an effective way to reduce the variance introduced by any single hash function producing an unusually high maximum trailing-zero count

4.4.1 Comparison with Spark's HyperLogLog

Spark's HyperLogLog implementation still outperforms our grouped FM estimate (9.1% vs 20.0% error), which is expected: HyperLogLog uses a more refined combination technique (harmonic mean combined with bias correction) than the simple stochastic averaging used here. HyperLogLog is now the industry standard for cardinality estimation on massive datasets, so the fact it outperformed the FM algorithm was to be expected (HyperLogLog was designed on top of the knowledge derived from FM). With that said, the project still shows how even a straightforward FM implementation with tunable parameters can produce an accurate cardinality estimation with only $O(k)$ memory.

4.5 Scaling to Massive Datasets

The FM algorithm is well suited for unbounded streams and massive datasets, since neither the memory footprint nor the processing cost of a single element depends on the number of distinct elements seen so far. As seen in the complexity analysis above, the algorithm requires only $O(k)$ space to maintain the trailing-zero registers, where k is the number of hash functions used; this stays constant regardless of whether the stream contains a thousand or a billion distinct *userIDs*.

As with the other probabilistic techniques discussed in this report (AMS and Bloom Filter), the trade-off for this scalability is accuracy: increasing *FM_NUM_HASHES* (and, as a result, the number of register groups used for stochastic averaging) reduces the expected estimation error, but at the cost of proportionally more memory and more hashing work per stream element. This tunable trade-off is precisely what allows FM, like AMS and the Bloom Filter, to scale to massive datasets where computing an exact answer would be infeasible.

Since each partition's registers depend only on the maximum trailing-zero count observed within that partition, partitions can be processed fully independently, with a final element-wise maximum across all partitions' registers (via *mapPartitions* and *reduce*) to obtain the global result. This requires no coordination between partitions until that final aggregation step, allowing FM to scale naturally across a distributed cluster as either data volume or worker count grows.

5 AMS Algorithm

The Alon–Matias–Szegedy (AMS) [2] algorithm is a probabilistic streaming algorithm used to estimate the frequency moments of a data stream without storing the entirety of said stream in memory.
In this project, it's used to estimate the second moment (F_2) of the stream of article sections referenced in each comment. The second moment is a very useful metric that we can use to understand how skewed the distribution of comments is across all possible sections. A uniform distribution over k sections would result in $F_2 \approx \frac{n^2}{k}$, while a more skewed distribution (where few sections dominate) would result in a much larger F_2 value.

5.1 Space/Time Complexity

The AMS algorithm maintains a fixed number of key-counter pairs (see AMS_STORED_VARS in Section 3) that is independent of the stream the algorithm operates on.
This means that, for a reservoir of size v, the space complexity for this algorithm is going to be $O(v)$. These characteristics make AMS suitable for streaming environments where the number of distinct elements is either large or unknown to the user before runtime.

5.2 Implementation Details

Each one of the stored v variables tracks two fundamental pieces of information:

- The stream element it is referring to.
- The number of times the same element was seen from its selected potition onward.

Let's now go over how the algorithm itself operates.

- The first v elements of the stream are used to initialize the reservoir (setting the count to 1 for each of them).
- For every subsequent element at position n (with $n > v$), the new position is selected with probability $\frac{v}{n}$.
  - If **not selected**: none of the v variables change which position they are tracking.
  - If **selected**: exactly **one** of the v variables is chosen uniformly at random and evicted, it is then replaced by a new variable tracking the current element, with its count reset to 1.
- Regardless of whether a variable was just evicted, every other variable's counter is incremented whenever the current stream element matches the value it is tracking.

The process above guarantees that, at any point in the stream, every position has an equal probability $\frac{v}{n}$ of being the position currently tracked by one of the v variables.

The code block below shows the function that implements the AMS algorithm in the submitted notebook.

python

```python
def ams_stream_counters(it, stored_vars=AMS_STORED_VARS, eval_every=None):
    import random
    rng = random.Random(RAND_SEED)
    stored = []  # variables
    counters = []  # counters for variables
    n = 0
    for element in it:
        n += 1
        replaced_idx = None
        if len(stored) < stored_vars:
            stored.append(element)
            counters.append(1)
            replaced_idx = len(stored) - 1
        else:
            if rng.random() < stored_vars / n:
                replaced_idx = rng.randrange(stored_vars)
                stored[replaced_idx] = element
                counters[replaced_idx] = 1

        for i in range(len(stored)):
            if i == replaced_idx:
                continue
            if stored[i] == element:
                counters[i] += 1

        # periodic yield for infinite streams
        if eval_every is not None and n % eval_every == 0:
            yield n, counters

    yield n, counters
```

5.3 Experimental Results

We evaluated the AMS algorithm against the stream of article sections referenced by comments, comparing its $\tilde{F}_2$ estimate against the exact F_2 value computed directly via aggregation over the same (sampled) dataset. The exact values obtained were:

Metric	Value
Stream length (n)	1,498,190
Exact F_2	533,168,431,742

We also computed the **skew ratio** of the comment distribution across sections, defined as the ratio between the observed F_2 and the F_2 expected under a uniform distribution over the same number of distinct sections (n^2/k, where k is the number of distinct sections). The uniform-case F_2 is approximately 53,442,220,860, giving a skew ratio of approximately **9.98x**. This confirms that comment activity is heavily concentrated in a small number of sections rather than spread evenly across all 42, consistent with *Opinion* being by far the most-commented section, as observed independently in Section 6 when selecting the section for the Bloom Filter experiment.

5.3.1 Choosing the Reservoir Size

Since AMS relies on randomised reservoir sampling, the accuracy of a single run depends on the variance of the estimator, which in turn depends on the reservoir size v. The original AMS paper [2] bounds the variance of a single estimator as:

$$\mathrm{Var}(X) \leq 2F_2^2$$

Averaging over v independent estimators reduces this variance by a factor of v, giving an expected relative standard error of approximately $\sqrt{\frac{2}{v}}$. Solving for the number of variables needed to reliably achieve a target relative error ε gives:

$$v \approx \frac{2}{\varepsilon^2}$$

For a target error of 10%, this predicts $v \approx 200$. To test this prediction directly, we swept *AMS_STORED_VARS* over $v \in [175, 225]$, bracketing the predicted optimum:

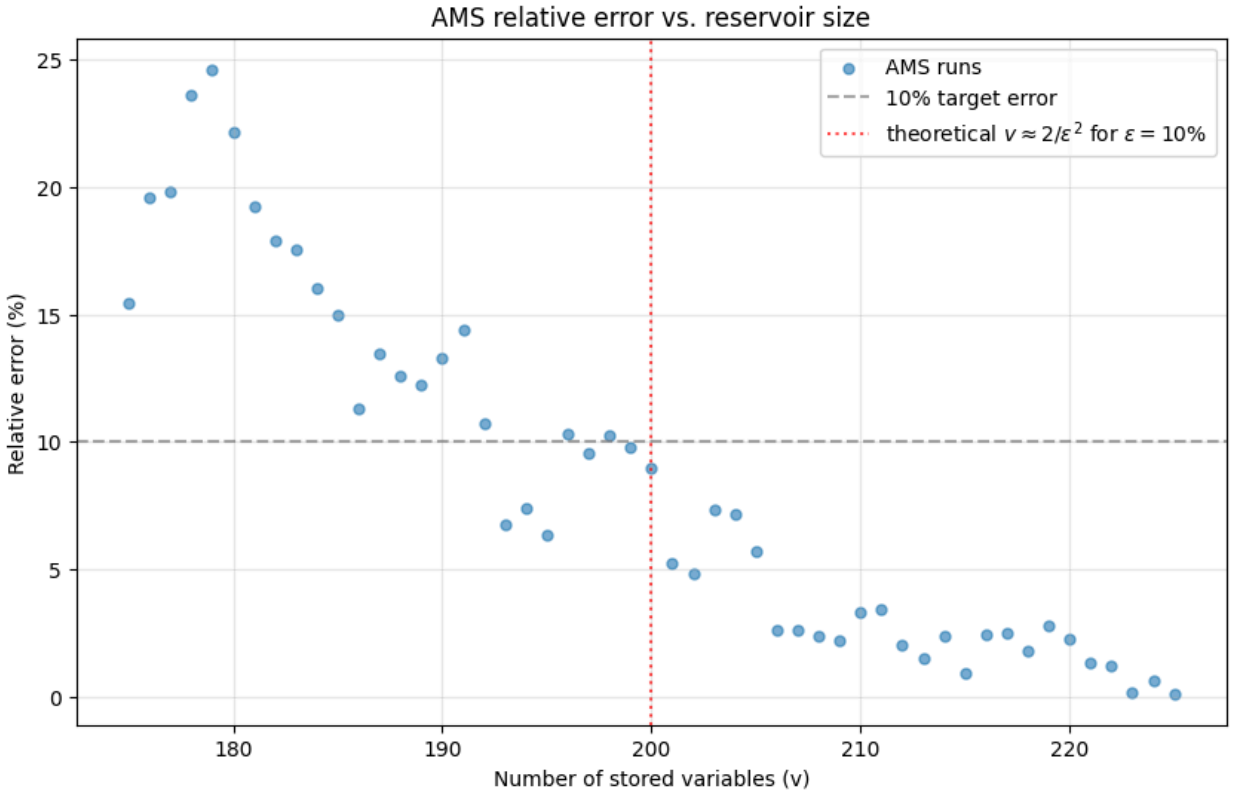

Figure 1: AMS relative error across a sweep of reservoir sizes ($v \in [175, 225]$), showing convergence toward low error as v approaches and exceeds the theoretically predicted $v \approx \frac{2}{\varepsilon^2} \approx 200$.

The results confirm the prediction closely. Averaging in blocks:

Range	Average error
$v \in [175, 192]$	16.6%
$v \in [193, 203]$	7.9%
$v \in [204, 225]$	2.2%

Error drops sharply as v approaches and exceeds the predicted $v \approx 200$, with several runs beyond this point achieving well under 1% error (e.g. 0.14% at $v = 223$, 0.11% at $v = 225$) exceeding what the theoretical bound alone would predict. This is consistent with $\mathrm{Var}(X) \leq 2F_2^2$ being a conservative, worst-case bound; the true variance for this dataset's actual frequency distribution appears more favourable than the generic bound assumes.

5.4 Scaling to Massive Datasets

The AMS algorithm is a good choice for massive datasets and unbounded streams, as long as the number of stored variables v is chosen accordingly to the desired accuracy.
As seen in Section 5.1, both the algorithm's space complexity and the processing cost per element are independent of the stream length (n) and the number of unique elements in the stream. As with all probabilistic approaches, the trade-off is accuracy, since AMS is a randomised estimator (the number of stored variables v has to grow in order to reduce variance).

If we think about it, the role of v is analogous to the number of hash functions (k) used in the FM algorithm explained in the first portion of the report.
Both these algorithms (and the Bloom Filter data structure below) trade space for accuracy in a way that can be tuned by the user, and this is exactly what makes them fit for massive datasets.

6 Bloom Filter

A bloom filter [3] is a probabilistic hash based data structure that is used for checking whether or not an element may be in the dataset while also being ok with having some false positives.
In this project we implemented this data structure to test it against the stream of unique *userIDs* of the people who left at least one comment on articles of the *Opinion* category (Section 2). More specifically, we started from the set of unique *userIDs* of the stream described above, we then fabricated n_{fake} new unique identifiers that are not present in the initial stream to use them to test for false positive rates at different configurations (more details in Section 6.3).

The bloom filter is structured as follows:

- An array of n bits is used to store information about the elements seen inside a stream or a generic dataset.
- k hash functions are used to map an element to k bit positions.
  - If the bit array already has all those bits set to one, then we may have already seen the element. We are still susceptible to false positives because, since the bit array size is finite, multiple hashes (mapped with the modulo inside the bit array) may end up resulting in the same bit positions being used.
  - If **at least one** of those bit positions is not 1, then the element was never encountered before (there can be no false negatives).

It is clear that the performance of the filter itself depends heavily on the number of bits used for the array and the number of hash functions used on every element. These parameters heavily depend on the use case and the available resources for the filter.

6.1 False Positives Theory

To understand why the bloom filter is used, we must also go into the math behind the number of false positives.
For the rest of this section, we are going to assume m as the number of bits and k as the number of hash functions used in the filter.
We are also going to assume that the selected hash functions map each element uniformely over the m bits.

- After a single element insertion, the probability that a specific bit is not going to be set to 1 as a result of the k hash functions is:
  - $P(\text{bit remains 0}) = \left(1 - \frac{1}{m}\right)^k$
- Given the probabilities described in the first point, for n insertions, the probability that a bit is not going to be set to 1 is:
  - $P(\text{bit remains 0 after n}) = \left(1 - \frac{1}{m}\right)^{kn}$

- The probability of a false positive then becomes:
  - $P(\text{false positive}) = \left(1 - \left(1 - \frac{1}{m}\right)^{kn}\right)^k$
- We can now approximate $\left(1 - \frac{1}{m}\right)^{kn}$ to $e^{\frac{-kn}{m}}$ to obtain:
  - $P(\text{false positive approx}) = \left(1 - e^{\frac{-kn}{m}}\right)^k$

We can see that, the higher m and k are, the less likely the filter is to report a false positive when it's being used.
It goes without saying that we cannot simply keep driving these numbers up without encountering memory and time issues. To correctly make use of a bloom filter, the user must choose m based on their memory constraints and k depending on the time that is allocatable to the task of computing the hash functions for a given element.

If we want to minimize the probability of false positives, we can choose the ideal number of hash functions k using the following equation and rounding the result to the closest integer:

- $k = \frac{m}{n} \ln(2)$

Where $\frac{m}{n}$ is the number of individual bits allocated for each element. This means that, even without knowing the exact value of n, we can use an estimation of it to choose a number of hash functions that is close to the ideal one.

This assumes the estimation is not too different from the actual value

6.2 Space/Time Complexity

When it comes to time complexity, both checking for an element and inserting a new one is equal to $O(k)$, where k is the number of hash functions used in the filter implementation. The time performance of a bloom filter varies greatly depending on the complexity of the hash functions used (which is generally tied to the complexity of the type of the elements to hash). Generalising this to a stream it becomes trivial that, for n elements, the complexity goes up to $O(nk)$. The space complexity of the bloom filter is trivial too, it is equal to $O(m)$ where m is the number of bits used.

6.3 Implementation Details

Implementing a bloom filter from scratch in python is very straightforward. Since python's integers are not restricted to a maximum amount of bits, we can use a single int as our bit array, this means that accessing individual bits can be done with simple bitwise operations that are very efficient. The class is required to have at least the following methods:

In order to use stable hash functions, we can store them in the object state as an array of k elements, the hash function chosen for this implementation is taken from the xxhash library.

Below is a code snippet to showcase the membership check and insertion methods for the described class:

```python
def add(self, x):
    self.bits |= self._map_bits(x)

def _map_bits(self, x):
    mask = 0
    for hf in self.hash_fns:
        mask |= (1 << (hf(x) % self.nbits))
    return mask

def contains(self, x):
    xmask = self._map_bits(x)
    return (self.bits & xmask) == xmask
```

- self.nbits: number of bits we want to store for the filter (m).
- self.hash_fns: array of k hash functions that return a 64 bit integer.
- self.bits: bit array that keeps track of the filter state (implemented using python's unbounded ints).

6.4 Chosen Task

As introduced in Section 6, we are going to test the proposed implementation in the following way:

- We first gather the set of all unique *UserIDs* of users that commented at least one article belonging to the *Opinion* section. This section was chosen because it was the one that presented the largest number of comments, making it a good fit for the experiment.
- We then generate a set of fake *UserIDs* that are guaranteed not to be found inside the first set. These IDs are generated using a range of integers that starts from the maximum ID found in the real set, the experiment is therefore easily reproducible since no randomness is added after the initial sampling.
- We "bootstrap" the filter by adding all the *UserIDs* of the first set to it, giving us a bitset inside the filter that we can test against fake IDs.
- We iterate over the second set to calculate the false positive rate with different configurations of m and k. The number of generated IDs is configurable via a global variable in the configuration section of the notebook.

The workflow we just described is going to allow us to compare the empirical results gathered with the theory explained in Section 6.1

6.5 Experimental Results

The following table contains the following information:

- The stars represent the k that resulted in the lower false positive rate for each configuration.
- The dashed vertical lines represent the theoretical best k as defined in Section 6.1.
- The full lines represent how the false positive rate evolves as we change k.
- The dashed lines that are overlapped to the full ones represent how the equations explaiend in Section 6.1 expected the behaviour to evolve.

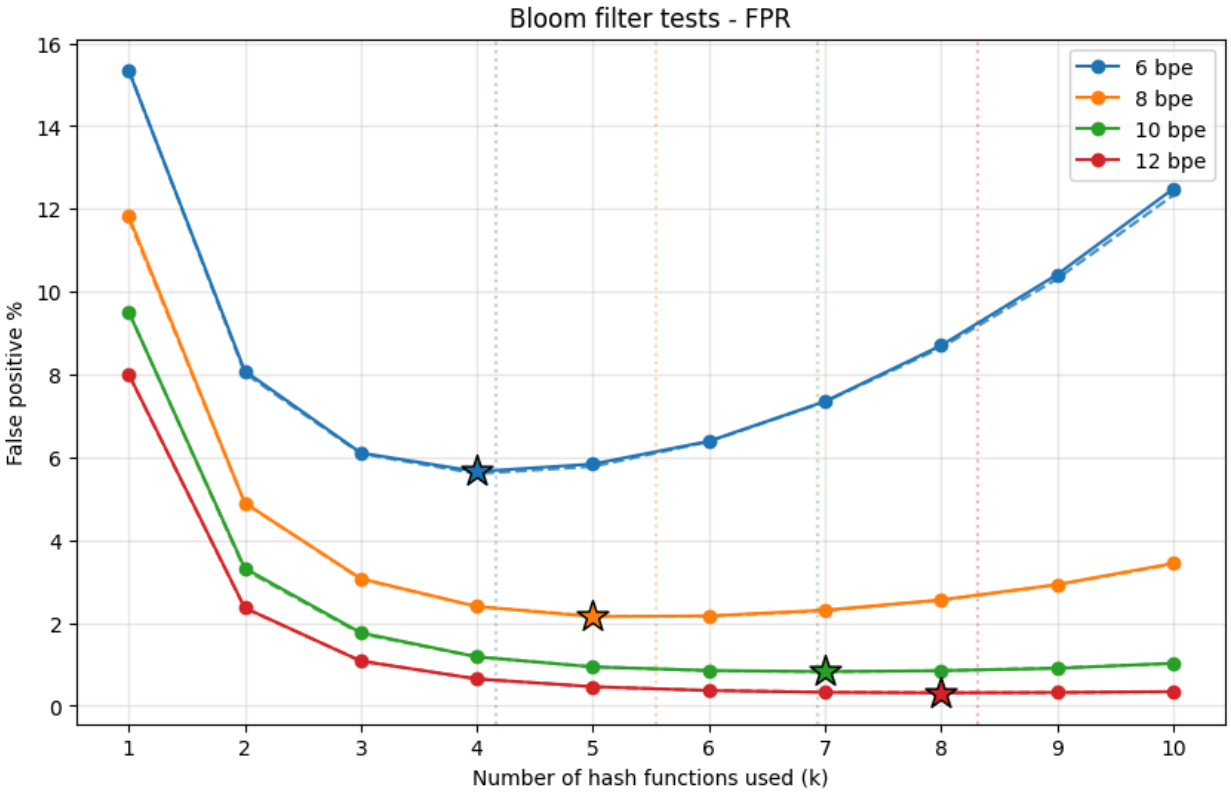

Figure 2: Bloom filter behavior when tested with a 10.0 fake IDs proportion

As we can see in the figure above, the empirical results closely follow the expected theoretical ones, confirming that the proposed implementation behaves as expected with the generated sample. The number of hash functions that led to the best result for each *bits per element* configuration are also very close to the theoretical ideal k described in Section 6.1.

6.6 Scaling to Massive Dataset

The bloom filter proposed in this project is suited for scaling to massive datasets, given that it is configured with m and k values that are appropriate for the task at hand.
Once the filter has been configured, the memory consumption is fixed at m bits, regardless of the number of elements inserted into the filter. Moreover, as seen in Section 6.2, the time complexity depends on the value of k; since this value is usually constant, the time complexity can be seen as $O(1)$ for each insertion and membership check.

On the other hand, approaches that use set-like data structures to keep track of the elements have a space complexity of $O(n)$. Additionally, as the number of elements inside a set grows, the performance of the set operations tends to degrade as well due to hash collisions (both on open and closed hashing approaches).

A bloom filter therefore provides a useful accuracy/memory tradeoff for large scale data processing. When the expected number of elements is known, the filter can be configured with $m = bn$ bits, where b is the number of bits allocated per element. In this case, the memory consumption also

grows linearly with the number of elements, but the required memory can be determined in advance and does not depend on the size of the individual elements.

7 Github Repository

The jupyer notebook submitted alongside this project and the typst source files for the report can be found here

8 Plagiarism and AI Usage Statement

I declare that this material, which I now submit for assessment, is entirely my own work and has not been taken from the work of others, save and to the extent that such work has been cited and acknowledged within the text of my work. I understand that plagiarism, collusion, and copying are grave and serious offences in the university and accept the penalties that would be imposed should I engage in plagiarism, collusion or copying. This assignment, or any part of it, has not been previously submitted by me or any other person for assessment on this or any other course of study. No generative AI tool has been used to write the code or the report content.

Bibliography

[1] F. Philippe and M. G. Nigel, "Probabilistic Counting Algorithms for Data Base Applications," *Journal of computer and system sciences 31*, 1985.

[2] N. Alon, Y. Matias, and M. Szegedy, "The Space Complexity of Approximating the Frequency Moments," *Journal of Computer and System Sciences*, vol. 58, no. 1, pp. 137–147, 1999, doi: https://doi.org/10.1006/jcss.1997.1545.

[3] B. H. Bloom, "Space/time trade-offs in hash coding with allowable errors," *Communications of the ACM*, vol. 13, no. 7, pp. 422–426, Jul. 1970, doi: 10.1145/362686.362692.

[4] B. Dornel, "New York Times Articles & Comments (2020)." 2021.